# 第五讲教学设计：Web应用的记忆核心—会话管理 (20分钟)

## 课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: Web应用的记忆核心：HttpSession会话管理
- **授课时长**: 20分钟
- **适用对象**: 已掌握Servlet和JSP基础，理解请求作用域（Request Scope）的学生

---

## 一、教学目标

### 知识目标

1. 理解HTTP协议的无状态性（Stateless）及其带来的挑战。
2. 掌握 `HttpSession` 对象的概念，理解其作为服务器端会话（Session）状态保持机制的原理。
3. 掌握获取 `HttpSession` 对象的方法： `request.getSession()` 。
4. 掌握在 `HttpSession` 中存储和检索数据的方法： `setAttribute()` 和 `getAttribute()` 。
5. 了解 `JSESSIONID` Cookie在会话跟踪（Session Tracking）中的作用。

### 能力目标

1. 能够利用 `HttpSession` 实现跨多个请求的用户登录状态保持。
2. 能够区分 `request` 作用域和 `session` 作用域的生命周期和适用场景。
3. 能够编写简单的登录拦截逻辑，防止未授权用户访问受保护资源。

### 素质目标

1. 培养严谨的逻辑思维，理解Web应用中"状态"的重要性。
2. 建立安全意识，理解会话管理是Web安全的基础。
3. **[思政融合]** 通过讲解HTTP的"无状态"和Session的"有状态"，类比个人与社会的关系。个人（请求）是独立的，但社会（应用）通过档案、身份等机制（Session）来记录和管理每个人的状态和历史，从而提供持续、稳定的服务。强调规则和契约（Cookie）在建立信任和维持秩序中的作用。

---

## 二、教学重点与难点

### 教学重点

- HTTP无状态性的直观体现（刷新/跳转后数据丢失）。
- `HttpSession` 的核心作用：为每个用户在服务器端开辟一块"专属内存空间"。
- `session.setAttribute()` 和 `session.getAttribute()` 的用法，实现数据跨请求共享。
- `request` 作用域与 `session` 作用域的根本区别。

### 教学难点

- **概念难点**: `HttpSession` 的生命周期管理（何时创建、何时销毁）。
- **原理难点**: 理解服务器是如何通过小小的 `JSESSIONID` Cookie自动将浏览器的每次请求与服务器上对应的 `HttpSession` 对象关联起来的。这个过程对开发者是透明的，但理解它至关重要。

---

## 三、教学内容与时间安排

### 第一部分：健忘的HTTP - 特工身份的丢失 (3分钟)

**内容要点**

1. **延续故事 (Continuing the Story):**
   - "上一节课，我们的特工 `007` 已经成功登录，并用EL和JSTL看到了一个非常清爽的任务列表页面 ( `mission.jsp` )。现在，他在任务页面点击了一个链接，想去查看他的个人档案 ( `profile.jsp` )。"
2. **制造问题 (Creating the Problem):**
   - **展示代码**:
     - `LoginServlet` : 将 `agentInfo` 对象存入 `request` 作用域，然后转发到 `mission.jsp` 。
     - `ProfileServlet` : 一个新的Servlet，它尝试从 `request` 作用域获取 `agentInfo` 。
   - **提问**: "当 `ProfileServlet` 执行时，它能成功拿到 `agentInfo` 对象吗？为什么？"
   - **引导学生回答**: 不能。因为HTTP请求是"一次性"的。 `LoginServlet` 处理的请求和 `ProfileServlet` 处理的请求是两次完全不同的请求。前一个请求的 `request` 对象在响应结束后就已经销毁了。
3. **点明本质 (Highlighting the Core Issue):**
   - 这就是 **HTTP协议的无状态性**。服务器默认是"健忘"的，它不记得上一个请求是谁发来的，干了什么。
   - "这很糟糕！就像你每次去银行办事，都得重新提交全套身份证明一样。我们需要一个方法，让服务器'记住'我们的特工 `007` 。"

**教学方法**

- 通过具体的用户场景和代码片段，让学生亲身体会"无状态"带来的问题。
- 设置悬念，激发学生寻找解决方案的兴趣。

### 第二部分：会话的诞生 - 服务器的记忆芯片 (8分钟)

**内容要点**

1. **引入解决方案：** `HttpSession`
   - **打个比方**: "为了解决健忘问题，服务器为每个首次到访的用户都分配一个'专属储物柜'，这就是 `HttpSession` 对象。用户在网站的整个访问期间，都可以使用这个柜子存取东西。"
2. **如何获得储物柜：** `request.getSession()`
   - **代码**: `HttpSession session = request.getSession();`
   - **讲解**: 这行代码是 `request.getSession(true)` 的简写。当 `true` 作为参数时，服务器会检查浏览器发来的请求里有没有"储物柜钥匙"。
     - **有钥匙**: 就用钥匙打开对应的柜子（返回已存在的Session）。
     - **没钥匙**: 就创建一个全新的柜子（创建新Session），并把新配的钥匙交给浏览器。
   - **重要变体**: `request.getSession(false)`
     - **作用**: 只获取，不创建。如果根据"钥匙"找到了对应的柜子，就返回它；如果找不到，不会创建新的，而是直接返回 `null` 。

    - **应用场景**: 这是实现**访问控制**的关键。在检查用户是否登录时，我们只关心他是否已经有会话，而不想为一个未经授权的访问创建新的会话。

3. **储物柜的"钥匙"：** `JSESSIONID`
    - **可视化**: 打开浏览器的开发者工具（F12 -> Application -> Cookies），展示那个名为 `JSESSIONID` 的Cookie。
    - **讲解**: 这就是服务器发给浏览器的"钥匙"。浏览器很聪明，之后每次访问这个网站，都会自动带上这把钥匙。服务器看到钥匙，就知道该去哪个柜子里找东西了。这个过程是自动的！
    - **技术补充：URL重写**
        - **提问**: 如果浏览器禁用了Cookie怎么办？
        - **解答**: 作为一种备选方案，服务器会将Session ID附加在URL的末尾，如 `.../profile.jsp;jsessionid=ABCDEF123456` 。这种技术称为URL重写。虽然现在已不常用，但了解它有助于我们完整地理解会话跟踪机制。

4. **使用储物柜：** `setAttribute` **和** `getAttribute`
    - **存东西**: `session.setAttribute("user", agentInfo);`
    - **取东西**: `AgentBean agent = (AgentBean) session.getAttribute("user");`
    - **强调**: 这和 `request` 的操作方法完全一样，只是对象从 `request` 换成了 `session` 。

**教学方法**

- **比喻法**: "储物柜和钥匙"的比喻贯穿始终，帮助学生形象化理解抽象概念。
- **工具演示**: 实际操作浏览器开发者工具，让学生眼见为实，增强信服力。

### 第三部分：实战 - 实现"一次登录，处处通行" (6分钟)

**内容要点**

1. **改造** `LoginServlet`
    - **Before**: `request.setAttribute("agentInfo", agent);`
    - **After**:

```
// ...验证用户名密码成功后...
HttpSession session = request.getSession(); // 获取或创建会话
session.setAttribute("loggedInAgent", agent); // 将特工信息存入会话
response.sendRedirect("mission.jsp"); // 重定向到任务页
```

2. **创建/改造** `ProfileServlet` **-> 升级为** `AuthFilter` **(更佳实践)**
    - **引入过滤器 (Filter)**: 在企业级应用中，我们不应在每个Servlet中都写一遍登录检查逻辑。更好的方法是创建一个过滤器，集中处理所有受保护资源的访问控制。
    - **关键逻辑**:

```
// AuthFilter.java
public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain) {
    HttpServletRequest request = (HttpServletRequest) req;
    HttpServletResponse response = (HttpServletResponse) res;
```

```java
    HttpSession session = request.getSession(false); // 只获取，不创建

    if (session != null && session.getAttribute("loggedInAgent") != null) {
        // 用户已登录，放行
        chain.doFilter(req, res);
    } else {
        // 用户未登录，重定向到登录页
        response.sendRedirect(request.getContextPath() + "/login.jsp");
    }
}
```

- **配置**: 在 `web.xml` 中配置此过滤器，使其拦截所有需要保护的路径（如 `/admin/*`）。

3. **实现登出功能（`LogoutServlet.java`）**
   - 将此功能从课后作业提升到课堂内容，形成闭环。
   - **代码实现**:

```java
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException {
        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate(); // 核心：销毁会话
        }
        response.sendRedirect("login.jsp"); // 重定向到登录页
    }
}
```

4. **JSP页面的变化**
   - EL表达式 `${loggedInAgent.codename}` 依然可以工作！因为EL会自动从page, request, **session**, application中搜索对象。

**教学方法**

- **代码重构**: 现场修改代码，解决第一部分提出的问题，形成闭环。
- **逻辑梳理**: 重点讲解 `if-else` 结构实现的登录状态判断和页面跳转逻辑。

### 第四部分：总结与展望（3分钟）

**总结要点**

1. **HTTP是无状态的**：这是Web工作的基本前提。
2. ** `HttpSession` 是"解药"**：它在服务器端为每个用户维护一个状态，生命周期贯穿用户的整个访问过程（会话）。
3. ** `JSESSIONID` 是"信使"**：它像一座桥梁，通过Cookie在客户端和服务器之间传递身份信息，关联起正确的会话。
4. **作用域对比**:

- **Request**: 一次请求的生命周期。像一次"快递"，送完即止。
- **Session**: 一次会话的生命周期。像一个"随身背包"，只要用户不离开（或超时），东西就一直在。

**Session生命周期详解**

- **创建**: 第一次调用 `request.getSession()` 或 `request.getSession(true)` 时。
- **销毁**:
  1. **手动销毁**: 调用 `session.invalidate()` 方法，通常用于"登出"功能。
  2. **超时销毁**: 用户在指定时间内没有任何操作，服务器会自动销毁该Session。默认通常是30分钟。
  3. **服务器关闭**: 应用停止或服务器关闭时。
- **超时配置**: 可以在 `web.xml` 中统一配置所有Session的超时时间（单位：分钟）。

```
<session-config>
    <session-timeout>20</session-timeout>
</session-config>
```

**展望 (The Next Step)**

- "现在，服务器能记住我们的特工了。但特工的任务列表、个人信息、装备清单...这些数据都存在Session里。如果服务器重启，或者用户量巨大，Session会变得臃肿不堪，内存会爆炸。Session里的数据是**临时的**。"
- "我们需要一个更可靠、更持久的地方来存储这些核心数据，一个不受服务器启停影响的'永久档案室'。"
- "下一节课，我们将学习如何将数据存入真正的'保险库'——**数据库**，并学习一种名为**DAO（Data Access Object）**的设计模式来优雅地操作它。"

---

## 四、教学方法与手段

**教学方法**

1. **故事驱动法 (Story-Driven)**: 延续"特工007"故事线
2. **问题导向法 (Problem-Oriented)**: 从HTTP无状态问题入手
3. **比喻教学法 (Analogy)**: "储物柜与钥匙"比喻Session机制
4. **代码实战法 (Practice)**: 实现登录保持和访问控制
5. **双语教学法 (Bilingual Teaching)**:
   - 课堂采用英文授课，关键技术术语提供中英文对照
   - 重要概念（Stateless, Session, Session Tracking, Cookie, Scope）首次出现时给出英文表达和中文释义
   - 鼓励学生使用英文术语描述会话管理机制
   - 代码注释和变量命名使用规范英文

**教学手段**

- IDE (IntelliJ IDEA/Eclipse)
- Tomcat服务器 (Tomcat Server)
- 浏览器开发者工具 (Browser DevTools)

- PPT/Marp课件 (Presentation Slides)

---

## 五、教学评价

- **过程性评价**: 能否说出 `request.getSession()` 和 `request.getSession(false)` 的区别；能否在浏览器中找到 `JSESSIONID` 。
- **终结性评价**: 能否独立编写一个需要登录才能访问的页面的后端逻辑；能否清晰地向他人解释Session的工作原理。

---

## 六、课后拓展

### 思考题

1. `session.invalidate()` 和 `session.removeAttribute("user")` 有什么本质区别？
2. 如果两个不同的浏览器（如Chrome和Edge）同时访问同一个Web应用，它们会共享同一个Session吗？为什么？

### 实践作业

1. **实现"记住我"功能**：研究Cookie的 `setMaxAge()` 方法，实现一个简单的"7天内免登录"功能。当用户勾选"记住我"并登录成功后，将一个包含用户身份标识的Cookie发送到客户端，并设置较长的有效期。
2. **完善访问控制**: 完善 `AuthFilter` ，使其可以配置一些"白名单"路径（如登录页面 `login.jsp` 、CSS、JS等静态资源），这些路径即使用户未登录也可以直接访问。